

Core Python for Agentic AI Development

1. Variables and Data Types

1. Definition

Variables are named containers used to store data values. Python is dynamically typed, so you don't need to declare the type explicitly.

2. Syntax and Explanation

```
variable_name = value
# Optional type hint
variable_name: type = value
```

3. Simple Real-World Explanation

A variable is like a labeled box in which you can store and update information whenever needed.

4. Coding Example

```
agent_name = "ResearchBot"
temperature = 0.7
max_tokens = 4096
is_active = True

print(agent_name, temperature)
```

5. Use Cases in Agentic AI Development

Storing agent configuration, current state, memory, goals, and tool settings.

2. Control Flow (if-elif-else, for, while)

1. Definition

Control flow statements allow the program to make decisions and repeat actions based on conditions.

2. Syntax and Explanation

```
if condition:
    # code
elif condition:
    # code
else:
    # code

for item in iterable:
    # code

while condition:
    # code
```

3. Simple Real-World Explanation

Like traffic signals and daily routines – deciding what to do next based on the current situation.

4. Coding Example

```
def agent_decision(confidence: float):
    if confidence > 0.85:
        return "Execute action"
    elif confidence > 0.5:
        return "Ask for more information"
    else:
        return "Replan goal"

print(agent_decision(0.92))
```

5. Use Cases in Agentic AI Development

Decision making, tool selection, handling different reasoning paths, and controlling agent loops.

3. List & Dictionary Comprehensions

1. Definition

A concise and Pythonic way to create lists and dictionaries by transforming and filtering data in a single line.

2. Syntax and Explanation

```
# List Comprehension
[expression for item in iterable if condition]
```

```
# Dict Comprehension
{key: value for item in iterable if condition}
```

3. Simple Real-World Explanation

Like quickly filtering good apples from a big basket and making a new basket in one go.

4. Coding Example

```
tools = ["web_search", "code", "invalid", "wikipedia"]

# Filter valid tools
valid_tools = [tool for tool in tools if tool != "invalid"]

print(valid_tools)
```

5. Use Cases in Agentic AI Development

Filtering tools, processing observations, cleaning LLM outputs, and creating memory summaries.

4. Functions, *args, **kwargs

1. Definition

Functions are reusable blocks of code. `*args` and `**kwargs` allow passing a variable number of arguments.

2. Syntax and Explanation

```
def function_name(required, default=10, *args, **kwargs):
    # *args → tuple
    # **kwargs → dictionary
```

3. Simple Real-World Explanation

A function is like a flexible worker who can accept different numbers of instructions and options.

4. Coding Example

```
def run_agent_step(goal, *tools, **context):
    print(f"Goal: {goal}")
    print(f"Tools: {tools}")
    print(f"Memory: {context.get('memory')}")
```

```
run_agent_step("Find latest AI news", "web_search", "news_api", memory="F
```

5. Use Cases in Agentic AI Development

Building flexible tool calling, agent execution loops, and passing dynamic context/memory.

5. Decorators

1. Definition

Decorators are functions that add extra functionality to other functions without modifying their original code.

2. Syntax and Explanation

```
def decorator(func):
    def wrapper(*args, **kwargs):
        # code before
        result = func(*args, **kwargs)
        # code after
        return result
    return wrapper

@decorator
def function():
    ...
```

3. Simple Real-World Explanation

Like adding a smartwatch to a person — it tracks time, logs activity, etc., without changing how the person works.

4. Coding Example

```
def log_step(func):
    def wrapper(*args, **kwargs):
        print(f"Starting: {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Finished: {func.__name__}")
        return result
    return wrapper

@log_step
def think(query):
```

```
return f"Thinking about: {query}"
```

```
think("How to build AI agents?")
```

5. Use Cases in Agentic AI Development

Logging agent steps, automatic retry logic, token/cost tracking, and safety validation.

6. Type Hints

1. Definition

Type hints are optional annotations that specify the expected data types of variables and function parameters.

2. Syntax and Explanation

```
from typing import List, Dict

def function(param: str) -> Dict[str, int]:
    ...
```

3. Simple Real-World Explanation

Like labeling kitchen jars (Sugar, Salt, Rice) so everyone knows what's inside without opening them.

4. Coding Example

```
from typing import List, Dict

def create_agent_memory(
    query: str,
    tools_used: List[str]
) -> Dict[str, str]:
    return {
        "query": query,
        "tools": ", ".join(tools_used),
        "status": "active"
    }
```

5. Use Cases in Agentic AI Development

Defining clean interfaces between agents, tools, and memory systems. Improves code reliability in large agent frameworks.

Data Structures, Error & File Handling

1. Lists

1. Definition

Lists are ordered, mutable (changeable) collections that can store items of different data types.

2. Syntax and Explanation

```
my_list = [item1, item2, item3]
# Methods: append(), extend(), pop(), remove(), sort(), etc.
```

3. Simple Real-World Explanation

A list is like a shopping list – you can add, remove, or change items easily and it maintains order.

4. Coding Example

```
tools = ["web_search", "code_execution", "wikipedia"]

tools.append("calculator")
tools.remove("wikipedia")

print(tools)
print(len(tools))
```

5. Use Cases in Agentic AI Development

Maintaining list of available tools, agent action history, task queues, and step-by-step observations.

2. Tuples

1. Definition

Tuples are ordered, immutable (unchangeable) collections, generally used for fixed data.

2. Syntax and Explanation

```
my_tuple = (item1, item2, item3)
# Cannot modify after creation
```

3. Simple Real-World Explanation

A tuple is like a fixed travel ticket with name, date, and seat number — once issued, it cannot be changed.

4.Coding Example

```
agent_config = ("ResearchBot", "gpt-4o", 0.7, True)

name, model, temp, active = agent_config

print(name, model)
```

5. Use Cases in Agentic AI Development

Storing fixed configurations, tool call signatures, coordinate data, and returning multiple values from functions safely.

3. Sets

1. Definition

Sets are unordered collections of unique items. They are mutable and support mathematical set operations.

2. Syntax and Explanation

```
my_set = {item1, item2, item3}
# Automatically removes duplicates
```

3. Simple Real-World Explanation

A set is like a guest list at an exclusive party — no duplicates allowed.

4.Coding Example

```
used_tools = {"web_search", "code_execution", "web_search"}

used_tools.add("wikipedia")

print(used_tools)          # No duplicates
print("code_execution" in used_tools)
```

5. Use Cases in Agentic AI Development

Tracking unique tools used in a session, avoiding duplicate API calls, and managing unique memory tags

or keywords.

4. Dictionaries

1. Definition

Dictionaries are unordered (ordered from Python 3.7+), mutable collections of key-value pairs for fast lookups.

2. Syntax and Explanation

```
my_dict = {"key1": value1, "key2": value2}
# Access: my_dict["key"] or my_dict.get("key")
```

3. Simple Real-World Explanation

A dictionary is like a real dictionary – you look up a word (key) to find its meaning (value) very quickly.

4. Coding Example

```
agent_state = {
    "name": "ResearchBot",
    "current_goal": "Find latest AI news",
    "tools_used": 3,
    "status": "running"
}

print(agent_state["current_goal"])
agent_state["tools_used"] += 1
```

5. Use Cases in Agentic AI Development

Storing agent memory, tool results, ReAct reasoning steps, configuration settings, and context windows.

5. Error Handling (try/except/finally)

1. Definition

Error handling allows your program to gracefully manage runtime errors instead of crashing.

2. Syntax and Explanation


```
try:
    # risky code
except Exception as e:
    # handle error
finally:
    # always runs (cleanup)
```

3. Simple Real-World Explanation

Like wearing a helmet while riding a bike – if something goes wrong, you are protected and can recover safely.

4. Coding Example

```
def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        return "Cannot divide by zero"
    except Exception as e:
        return f"Error: {e}"
    finally:
        print("Operation completed")

print(safe_divide(10, 0))
```

5. Use Cases in Agentic AI Development

Handling API failures, tool errors, JSON parsing issues, and ensuring agents continue running even when one tool fails.

6. File Handling (Text, JSON, CSV, Binary)

1. Definition

File handling involves reading from and writing to files on disk in different formats.

2. Syntax and Explanation

```
# Text
with open("file.txt", "r") as f:
    data = f.read()
```

```
# JSON
import json
json.dump(data, file)
data = json.load(file)
```

3. Simple Real-World Explanation

Like saving your important documents, notes, or photos in folders so you can access or update them later.

4. Coding Example

```
import json

# Save agent memory
memory = {"goal": "Learn Python", "progress": "Part 3 completed"}

with open("agent_memory.json", "w") as f:
    json.dump(memory, f, indent=2)

# Read it back
with open("agent_memory.json", "r") as f:
    loaded = json.load(f)

print(loaded)
```

5. Use Cases in Agentic AI Development

Saving/loading agent memory, conversation history, tool outputs, logs, configurations, and long-term learning data.

Object-Oriented Python

1. Classes and `__init__`

1. Definition

A class is a blueprint for creating objects. The `__init__` method is a special constructor that runs automatically when a new object is created.

2. Syntax and Explanation

```
class ClassName:
    def __init__(self, parameters):
        self.attribute = parameters
```

3. Simple Real-World Explanation

A class is like a blueprint for a car. `__init__` is the process of assembling the car with specific features when you order one.

4. Coding Example

```
class AIAgent:
    def __init__(self, name, role):
        self.name = name
        self.role = role
        self.memory = []

    def introduce(self):
        print(f"I am {self.name}, a {self.role} agent.")

agent = AIAgent("ResearchBot", "Information Gatherer")
agent.introduce()
```

5. Use Cases in Agentic AI Development

Creating different types of agents (Research Agent, Planner Agent, Executor Agent) with their own properties and memory.

2. Instance vs Class Methods

1. Definition

Instance methods work with individual objects. Class methods work with the class itself and are shared across all instances.

2. Syntax and Explanation

```
class MyClass:
    def instance_method(self):          # Works with instance
        ...

    @classmethod
    def class_method(cls):             # Works with class
        ...
```

3. Simple Real-World Explanation

Instance method = actions of one specific car.

Class method = actions for the entire car factory.

4. Coding Example

```
class AIAgent:
    total_agents = 0  # Class attribute

    def __init__(self, name):
        self.name = name
        AIAgent.total_agents += 1

    def instance_info(self):  # Instance method
        print(f"I am {self.name}")

    @classmethod
    def show_total_agents(cls):  # Class method
        print(f"Total agents created: {cls.total_agents}")

agent1 = AIAgent("Satyasai")
agent2 = AIAgent("ResearchBot")
AIAgent.show_total_agents()
```

5. Use Cases in Agentic AI Development

Tracking total number of active agents, managing shared configurations, and creating factory methods for agent creation.

3. Inheritance

1. Definition

Inheritance allows a new class (child) to inherit attributes and methods from an existing class (parent).

2. Syntax and Explanation

```
class Parent:
    ...

class Child(Parent):
    ...
```

3. Simple Real-World Explanation

Like a child inheriting traits from parents but also having their own unique qualities.

4.Coding Example

```
class Agent:
    def __init__(self, name):
        self.name = name

    def think(self):
        print(f"{self.name} is thinking...")

class ResearchAgent(Agent):
    def search(self):
        print(f"{self.name} is searching the web...")

ra = ResearchAgent("DeepSearch")
ra.think()          # Inherited method
ra.search()         # New method
```

5. Use Cases in Agentic AI Development

Creating specialized agents (ResearchAgent, PlannerAgent, CriticAgent) from a base Agent class.

4. Encapsulation

1. Definition

Encapsulation is the concept of restricting direct access to some of an object's components and bundling data and methods together.

2. Syntax and Explanation

```
class MyClass:
    def __init__(self):
        self.public = "visible"
        self._protected = "semi-visible"
        self.__private = "hidden"      # Name mangling
```

3. Simple Real-World Explanation

Like a medicine capsule – the outer shell protects the medicine inside, and you only interact with it through proper ways.

4.Coding Example

```

class SecureAgent:
    def __init__(self, name):
        self.name = name
        self.__api_key = "secret123"          # Private

    def get_api_key(self):                    # Controlled access
        return "****" + self.__api_key[-4:]

agent = SecureAgent("SecureBot")
print(agent.name)
print(agent.get_api_key())

```

5. Use Cases in Agentic AI Development

Protecting API keys, internal memory, and sensitive agent states from accidental modification.

5. Polymorphism

1. Definition

Polymorphism allows objects of different classes to be treated as objects of a common parent class, with each class implementing the same method differently.

2. Syntax and Explanation

```

# Same method name, different behavior
obj.method()

```

3. Simple Real-World Explanation

Like pressing the "Start" button on different devices – a fan starts spinning, a car starts the engine, a computer boots up.

4. Coding Example

```

class Agent:
    def act(self):
        pass

class ResearchAgent(Agent):
    def act(self):
        print("Searching information...")

class ExecutionAgent(Agent):

```

```
def act(self):
    print("Executing tool...")

agents = [ResearchAgent(), ExecutionAgent()]

for agent in agents:
    agent.act()          # Same method, different behavior
```

5. Use Cases in Agentic AI Development

Creating flexible multi-agent systems where different agents can be called uniformly using the same interface.

6. DataClasses

1. Definition

DataClasses are a decorator that automatically adds special methods like `__init__`, `__repr__`, and `__eq__` to classes that primarily store data.

2. Syntax and Explanation

```
from dataclasses import dataclass

@dataclass
class Person:
    name: str
    age: int
```

3. Simple Real-World Explanation

A quick and clean way to create simple data containers without writing boilerplate code.

4. Coding Example

```
from dataclasses import dataclass
from typing import List

@dataclass
class AgentStep:
    thought: str
    action: str
    action_input: str
    observation: str = None
```

```
step = AgentStep(  
    thought="I need weather info",  
    action="web_search",  
    action_input="Hyderabad weather"  
)  
print(step)
```

5. Use Cases in Agentic AI Development

Representing agent steps, tool calls, memory entries, and ReAct-style reasoning traces cleanly and efficiently.

Python Programming Problems

Section 1: Core Python (Questions 1-8)

1. Create variables to store your name (string), age (integer), height (float), and whether you are a student (boolean). Print all of them with proper labels.
 2. Write a program that takes a number as input and uses `if-elif-else` to check whether the number is positive, negative, or zero.
 3. Write a `for` loop to print the multiplication table of a given number (e.g., 5).
 4. Use list comprehension to create a list of squares of numbers from 1 to 20.
 5. Use dictionary comprehension to create a dictionary where keys are numbers from 1 to 10 and values are their cubes.
 6. Write a function that takes a name and greeting as parameters and returns a greeting message. Also add a default value for greeting.
 7. Create a simple decorator that prints `"Function started"` before and `"Function ended"` after any function runs.
 8. Write a function with type hints that takes a list of integers and returns their average as a float.
-

Section 2: Data Structures, Error Handling & File Handling (Questions 9-17)

9. Create a list of 8 fruits. Add 2 more fruits, remove 1 fruit, and print the final list.
10. Create a tuple with your favorite 5 foods. Unpack the tuple into 5 separate variables.
11. Given a list `[1, 2, 2, 3, 4, 4, 5]`, convert it into a set and print its length.

12. Create a dictionary to store 5 students and their marks. Print the student with the highest marks.
13. Write a function that takes two numbers and handles `ZeroDivisionError` using try-except.
14. Write a program that reads a JSON file named `config.json` (create sample content) and prints its contents safely with error handling.
15. Create a list of numbers from 1 to 50 using list comprehension, then use another comprehension to filter only even numbers.
16. Write a function using `*args` and `**kwargs` that prints all positional and keyword arguments passed to it.
17. Create a simple `AIAgent` class with attributes `name`, `role`, and `memory` (list). Add methods to add memory and show memory. Use proper type hints.

Section 3: Object-Oriented Python (Questions 18-25)

18. Create a `Person` class with `__init__` method that takes `name` and `age`. Add a method `introduce()` that prints the person's details.
 19. Create a class `BankAccount` with a class variable `total_accounts` that keeps count of how many accounts are created. Use a class method to show the total.
 20. Implement single inheritance: Create a `Vehicle` parent class and a `Car` child class that inherits from it. Add one extra method in the `Car` class.
 21. Create a class `Student` where the email attribute is private (`__email`). Provide a method to view the email(encapsulation).
 22. Demonstrate polymorphism: Create two classes `Dog` and `Cat` that both have a method `speak()`, but they print different sounds.
 23. Use `@dataclass` to create a `Book` class with attributes: title, author, and year.
 24. Create a class `Calculator` with an instance method `add()` and a class method `show_version()`.
 25. Create a base class `Shape` with a method `area()`. Create two child classes `Rectangle` and `Circle` that implement their own `area()` method (Polymorphism).
-